```
// This Pine Script® code is subject to the terms of the Mozilla Public License 2.0 at https://
mozilla.org/MPL/2.0/
// © officialjackofalltrades
//@version=6
indicator("Order Block Engine [JOAT]", shorttitle="OB [JOAT]", overlay=true,
max_lines_count=500, max_labels_count=500, max_boxes_count=500)

//
═══════════════════════════════════════════════════════════════════════════════

//  ORDER BLOCK ENGINE [JOAT]
//  Institutional order blocks — the last opposing-close candle before a
//  displacement leg that BREAKS a confirmed swing by more than displacement×ATR,
//  with volume confirmation. Only a few unmitigated, high-grade zones survive.
//  Zones extend right until mitigated (then stop/grey). A clean LABEL pill fires
//  on a valid retest + reaction, with a full R-multiple TP/SL zone construct.
//  Grey / white + blue chrome identity.
//
═══════════════════════════════════════════════════════════════════════════════

// ──────────────────────────────────────────────────────── 01 · ENGINE
gEng = "01 · Engine"
atrLen     = input.int(14, "ATR Length", minval=1, group=gEng, tooltip="Volatility basis for
displacement, grading and SL/TP placement. Adapts to any asset / timeframe.")
pivotLen   = input.int(8, "Structure Pivot", minval=2, maxval=50, group=gEng, tooltip="Bars either
side of a swing pivot. The impulse must break the most recent confirmed swing to qualify.")
dispMult   = input.float(0.5, "Displacement Break (× ATR)", minval=0.05, maxval=5.0, step=0.05,
group=gEng, tooltip="The breaking candle must close beyond the swing by more than this
multiple of ATR. Higher = stricter = fewer, cleaner blocks.")
obSearch   = input.int(10, "OB Candle Search", minval=1, maxval=40, group=gEng, tooltip="How
far back from the break to find the last opposing-close candle that becomes the order block.")
allowBull  = input.bool(true, "Detect Bullish OBs", group=gEng)
allowBear  = input.bool(true, "Detect Bearish OBs", group=gEng)

// ──────────────────────────────────────────────────────── 02 · FILTERS
gFilt = "02 · Filters"
useVol     = input.bool(true, "Require Volume Expansion", group=gFilt, tooltip="Breaking candle
volume must exceed its baseline. Auto-skipped on symbols with no volume.")
volLen     = input.int(20, "Volume Baseline Length", minval=2, maxval=200, group=gFilt)
volMult    = input.float(1.3, "Volume Multiplier", minval=0.1, maxval=5.0, step=0.05, group=gFilt)
minGrade   = input.float(5.0, "Minimum Grade To Signal (0-10)", minval=0.0, maxval=10.0,
step=0.5, group=gFilt, tooltip="Order blocks scoring below this never emit a BUY / SELL signal.")
confirmClose = input.bool(true, "Confirm On Bar Close", group=gFilt, tooltip="Evaluate detection /
mitigation / signals on confirmed bars only, to avoid intrabar repaint.")

// ──────────────────────────────────────────────────────── 03 · ZONES
gVis = "03 · Zones"
showZones  = input.bool(true, "Show OB Zones", group=gVis)
maxKeep    = input.int(6, "Max Zones Kept", minval=1, maxval=20, group=gVis, tooltip="Only the
most recent N order-block zones are drawn — keeps the chart clean (4-6 recommended).")
extendBars = input.int(50, "Zone Extension (bars)", minval=5, maxval=400, group=gVis)
boxTransp  = input.int(85, "Zone Fill Transparency", minval=50, maxval=98, group=gVis)
showGrade  = input.bool(true, "Show ★ Grade Label", group=gVis, tooltip="A single ★-grade
(0-10) tag on each zone. No duplicate labels.")
```

```
keepMit    = input.bool(true, "Keep Mitigated Zones (grey)", group=gVis, tooltip="Mitigated /
invalidated zones freeze and turn grey instead of being deleted.")
bullCol    = input.color(#4A90E2, "Bullish OB", group=gVis, inline="c1")
bearCol    = input.color(#78909C, "Bearish OB", group=gVis, inline="c1")
lblSize    = input.string("Small", "Label Size", options=["Tiny", "Small", "Normal", "Large"],
group=gVis)

// ──────────────────────────────────────────────────────────── 04 · SIGNALS & RISK
──────────────────────────────────────────────

gRisk = "04 · Signals & Risk"
showSignals= input.bool(true, "Show BUY / SELL Pills", group=gRisk)
showSLTP   = input.bool(true, "Draw TP / SL Zone", group=gRisk)
slBuf      = input.float(0.25, "Stop Buffer Beyond Zone (× ATR)", minval=0.0, maxval=3.0,
step=0.05, group=gRisk)
tp1R       = input.float(1.5, "TP1 (R multiple)", minval=0.2, maxval=20, step=0.1, group=gRisk)
tp2R       = input.float(3.0, "TP2 (R multiple)", minval=0.2, maxval=30, step=0.1, group=gRisk)
riskExtend = input.int(30, "Trade Zone Projection (bars)", minval=5, maxval=200, group=gRisk)
maxTrades  = input.int(6, "Max TP/SL Sets Kept", minval=1, maxval=40, group=gRisk)

// ──────────────────────────────────────────────────────────── 05 · EXTRAS
──────────────────────────────────────────────────

gExt = "05 · Extras"
colorCandles = input.bool(false, "Colored Zone-Reader Candles", group=gExt, tooltip="Tints
candles that trade inside an active OB zone, and on signals. Off for a clean default.")
useVwap    = input.bool(false, "Show VWAP + σ Band", group=gExt)
vwapDev    = input.float(1.0, "Band σ", minval=0.5, maxval=4.0, step=0.5, group=gExt)
vwapLen    = input.int(100, "σ Length", minval=10, maxval=500, group=gExt)

// ──────────────────────────────────────────────────────────── 06 · DASHBOARD
──────────────────────────────────────────────

gDash = "06 · Dashboard"
showDash   = input.bool(true, "Show Dashboard", group=gDash)
dashPos    = input.string("Top Right", "Position", options=["Top Right", "Top Left", "Bottom
Right", "Bottom Left", "Middle Right"], group=gDash)
dashSize   = input.string("Normal", "Text Size", options=["Tiny", "Small", "Normal"], group=gDash)

// ──────────────────────────────────────────────────────────── PALETTE (grey / white + blue
chrome) ──────────
color chrome   = #B0BEC5
color accent   = #4A90E2
color txtHi    = #ECEFF1
color txtLo    = #90A4AE
color greyMit  = #5A6670
color redRisk  = #E05252
color panelBg  = #12161B
color panelAlt = #1A2129
color hdrDark  = #0E1013
color bullFill = color.new(bullCol, boxTransp)
color bearFill = color.new(bearCol, boxTransp)

// ──────────────────────────────────────────────────────────── PURE HELPERS
──────────────────────────────────────────────

f_size(string s) =>
    switch s
        "Tiny"   => size.tiny
        "Small"  => size.small
        "Normal" => size.normal
        "Large"  => size.large
        => size.small
```

```
f_pos(string p) =>
    switch p
        "Top Left"     => position.top_left
        "Top Right"    => position.top_right
        "Bottom Left"  => position.bottom_left
        "Bottom Right" => position.bottom_right
        "Middle Right" => position.middle_right
        => position.top_right

f_stars(float sc) => sc >= 9.0 ? "★★★★★" : sc >= 7.5 ? "★★★★" : sc >= 6.0 ? "★★★" : sc
>= 4.5 ? "★★" : sc >= 2.5 ? "★" : "·"
f_tier(float sc)  => sc >= 9.0 ? "ELITE" : sc >= 7.5 ? "STRONG" : sc >= 6.0 ? "SOLID" : sc >= 4.5 ?
"FAIR" : "WEAK"

f_grade(float dispAtr, float volRatio, float bodyRatio) =>
    s = 0.0
    s += dispAtr >= 2.0 ? 4.0 : dispAtr >= 1.4 ? 3.2 : dispAtr >= 1.0 ? 2.5 : dispAtr >= 0.6 ? 1.7 : 1.0
    s += volRatio >= 2.2 ? 3.5 : volRatio >= 1.6 ? 2.7 : volRatio >= 1.2 ? 1.8 : volRatio >= 0.9 ? 1.0 :
0.4
    s += bodyRatio >= 0.8 ? 2.5 : bodyRatio >= 0.6 ? 1.8 : bodyRatio >= 0.4 ? 1.0 : 0.4
    math.min(10.0, math.max(0.0, s))

// ———————————————————————————————————————————— TYPES
————————————————————————————————————————————————————————————————————

type OB
    int   dir
    float top
    float bot
    float grade
    int   madeBar
    bool  mit
    box   bx
    label lb

type Trade
    box   rew
    box   rsk
    line  en
    line  sl
    line  t1
    line  t2
    label enl
    label sll
    label t1l
    label t2l
    label rewl
    label rskl

// ———————————————————————————————————————————— STATE
————————————————————————————————————————————————————————————————————

var array<OB>    obs    = array.new<OB>()
var array<Trade> trades = array.new<Trade>()
var array<label> sigLbls = array.new<label>()

var float swingHigh = na
var float swingLow  = na
var bool  shBroken  = true
var bool  slBroken  = true

var int   biasDir    = 0
```

```
var int   totBuy     = 0
var int   totSell    = 0
var int   wins       = 0
var int   losses     = 0
var int   lastMitDir = 0
var float lastMitLvl = na
var float lastMitGr  = na

// active trade
var bool  tActive = false
var int   tSide   = 0
var int   tBar    = na
var float tSL     = na
var float tT2     = na
var box   aRew = na
var box   aRsk = na
var line  aEn  = na
var line  aSl  = na
var line  aT1  = na
var line  aT2  = na
var label aEnL = na
var label aSlL = na
var label aT1L = na
var label aT2L = na
var label aRewL = na
var label aRskL = na

// ———————————————————————————————————————— DRAWING HELPERS ————————————————————————————————————————

f_retire(box bx, label lb) =>
    if not na(bx)
        box.set_bgcolor(bx, color.new(greyMit, 90))
        box.set_border_color(bx, color.new(greyMit, 55))
        box.set_right(bx, bar_index)
    if not na(lb)
        label.set_color(lb, color.new(greyMit, 35))
        label.set_textcolor(lb, color.new(greyMit, 10))

f_kill(box bx, label lb) =>
    if not na(bx)
        box.delete(bx)
    if not na(lb)
        label.delete(lb)

f_close_ob(OB ob) =>
    if keepMit
        f_retire(ob.bx, ob.lb)
    if not keepMit
        f_kill(ob.bx, ob.lb)
        ob.bx := na
        ob.lb := na

f_pushOB(OB ob) =>
    array.push(obs, ob)
    while array.size(obs) > maxKeep
        OB old = array.shift(obs)
        if not na(old.bx)
            box.delete(old.bx)
        if not na(old.lb)
            label.delete(old.lb)
```

```
// ──────────────────────────────────────────────────────────── SERIES
──────────────────────────────────────────────────────────────────────────────
float atr    = ta.atr(atrLen)
float vol    = nz(volume)
float volSma = ta.sma(vol, volLen)
float volR   = volSma > 0 ? vol / volSma : 1.0
float ph     = ta.pivothigh(pivotLen, pivotLen)
float pl     = ta.pivotlow(pivotLen, pivotLen)
float bodyR  = (high - low) > 0 ? math.abs(close - open) / (high - low) : 0.0
bool  confBar = not confirmClose or barstate.isconfirmed
bool  volOk  = not useVol or volSma <= 0 or volR >= volMult

// ──────────────────────────────────────────────────────────── SWING STRUCTURE
──────────────────────────────────────────────────

if not na(ph)
    swingHigh := ph
    shBroken  := false
if not na(pl)
    swingLow := pl
    slBroken := false

// ──────────────────────────────────────────────────────────── DISPLACEMENT / OB
CREATION ──────────────────────────────
bool bosUp = allowBull and confBar and not na(swingHigh) and not shBroken and not na(atr) and
atr > 0 and close > open and (close - swingHigh) > dispMult * atr and volOk
bool bosDn = allowBear and confBar and not na(swingLow) and not slBroken and not na(atr) and
atr > 0 and close < open and (swingLow - close) > dispMult * atr and volOk

if bosUp
    shBroken := true
    biasDir  := 1
    int obIdx = -1
    for i = 1 to obSearch
        if close[i] < open[i]
            obIdx := i
            break
    if obIdx > 0
        float top = high[obIdx]
        float bot = low[obIdx]
        float dispAtr = (close - swingHigh) / atr
        float gr = f_grade(dispAtr, volR, bodyR)
        int leftBar = bar_index - obIdx
        box bx = na
        label lb = na
        if showZones
            bx := box.new(leftBar, top, bar_index + extendBars, bot, xloc=xloc.bar_index,
bgcolor=bullFill, border_color=color.new(bullCol, 35), border_width=1)
            if showGrade
                lb := label.new(leftBar, top, f_stars(gr) + " " + str.tostring(gr, "#.#"), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_down, color=color.new(bullCol, 15), textcolor=color.white,
size=f_size(lblSize))
        f_pushOB(OB.new(1, top, bot, gr, bar_index, false, bx, lb))

if bosDn
    slBroken := true
    biasDir  := -1
    int obIdx = -1
    for i = 1 to obSearch
        if close[i] > open[i]
```

```
            obIdx := i
            break
    if obIdx > 0
        float top = high[obIdx]
        float bot = low[obIdx]
        float dispAtr = (swingLow - close) / atr
        float gr = f_grade(dispAtr, volR, bodyR)
        int leftBar = bar_index - obIdx
        box bx = na
        label lb = na
        if showZones
            bx := box.new(leftBar, top, bar_index + extendBars, bot, xloc=xloc.bar_index,
bgcolor=bearFill, border_color=color.new(bearCol, 35), border_width=1)
            if showGrade
                lb := label.new(leftBar, bot, f_stars(gr) + " " + str.tostring(gr, "#.#"), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_up, color=color.new(bearCol, 15), textcolor=color.white,
size=f_size(lblSize))
        f_pushOB(OB.new(-1, top, bot, gr, bar_index, false, bx, lb))

// ──────────────────────────────────────────────────────────── MANAGE / MITIGATE /
SIGNAL ──────────────────────────────
bool  buySig    = false
bool  sellSig   = false
float sigTop    = na
float sigBot    = na
float sigGrade  = na
bool  inBullZone = false
bool  inBearZone = false

if array.size(obs) > 0
    for i = 0 to array.size(obs) - 1
        OB ob = array.get(obs, i)
        if not ob.mit
            if not na(ob.bx)
                box.set_right(ob.bx, bar_index + extendBars)
            if close <= ob.top and close >= ob.bot
                if ob.dir == 1
                    inBullZone := true
                else
                    inBearZone := true
            if confBar and bar_index > ob.madeBar
                if ob.dir == 1
                    if close < ob.bot
                        ob.mit := true
                        lastMitDir := 1
                        lastMitLvl := ob.bot
                        lastMitGr  := ob.grade
                        f_close_ob(ob)
                    else if low <= ob.top and close > ob.bot and close > open and ob.grade >= minGrade
                        buySig := true
                        sigTop := ob.top
                        sigBot := ob.bot
                        sigGrade := ob.grade
                        ob.mit := true
                        lastMitDir := 1
                        lastMitLvl := ob.top
                        lastMitGr  := ob.grade
                        f_close_ob(ob)
                else
                    if close > ob.top
```

```
                    ob.mit := true
                    lastMitDir := -1
                    lastMitLvl := ob.top
                    lastMitGr  := ob.grade
                    f_close_ob(ob)
                else if high >= ob.bot and close < ob.top and close < open and ob.grade >=
minGrade
                    sellSig := true
                    sigTop := ob.top
                    sigBot := ob.bot
                    sigGrade := ob.grade
                    ob.mit := true
                    lastMitDir := -1
                    lastMitLvl := ob.bot
                    lastMitGr  := ob.grade
                    f_close_ob(ob)

// one signal per bar, long has priority
sellSig := sellSig and not buySig
if buySig
    totBuy += 1
if sellSig
    totSell += 1

// ———————————————————————————————————————————— ACTIVE TRADE — EXTEND /
FREEZE ————————————————
if tActive
    int rN = math.min(bar_index, nz(tBar, bar_index) + riskExtend)
    if not na(aRew)
        box.set_right(aRew, rN)
    if not na(aRsk)
        box.set_right(aRsk, rN)
    if not na(aEn)
        line.set_x2(aEn, rN)
    if not na(aSl)
        line.set_x2(aSl, rN)
    if not na(aT1)
        line.set_x2(aT1, rN)
    if not na(aT2)
        line.set_x2(aT2, rN)

    bool slHit = tSide == 1 ? low <= tSL : high >= tSL
    bool tpHit = tSide == 1 ? high >= tT2 : low <= tT2
    if slHit or tpHit
        int fE = math.max(nz(tBar, bar_index) + 2, bar_index)
        if not na(aRew)
            box.set_right(aRew, fE)
        if not na(aRsk)
            box.set_right(aRsk, fE)
        if not na(aEn)
            line.set_x2(aEn, fE)
        if not na(aSl)
            line.set_x2(aSl, fE)
        if not na(aT1)
            line.set_x2(aT1, fE)
        if not na(aT2)
            line.set_x2(aT2, fE)
        if not na(aEnL)
            label.set_x(aEnL, fE)
        if not na(aSlL)
```

```
            label.set_x(aSlL, fE)
        if not na(aT1L)
            label.set_x(aT1L, fE)
        if not na(aT2L)
            label.set_x(aT2L, fE)
        if tpHit and not slHit
            wins += 1
        else
            losses += 1
        array.push(trades, Trade.new(aRew, aRsk, aEn, aSl, aT1, aT2, aEnL, aSlL, aT1L, aT2L,
aRewL, aRskL))
        while array.size(trades) > maxTrades
            Trade od = array.shift(trades)
            if not na(od.rew)
                box.delete(od.rew)
            if not na(od.rsk)
                box.delete(od.rsk)
            if not na(od.en)
                line.delete(od.en)
            if not na(od.sl)
                line.delete(od.sl)
            if not na(od.t1)
                line.delete(od.t1)
            if not na(od.t2)
                line.delete(od.t2)
            if not na(od.enl)
                label.delete(od.enl)
            if not na(od.sll)
                label.delete(od.sll)
            if not na(od.t1l)
                label.delete(od.t1l)
            if not na(od.t2l)
                label.delete(od.t2l)
            if not na(od.rewl)
                label.delete(od.rewl)
            if not na(od.rskl)
                label.delete(od.rskl)
        tActive := false
        tSide := 0
        tBar := na
        tSL := na
        tT2 := na
        aRew := na
        aRsk := na
        aEn := na
        aSl := na
        aT1 := na
        aT2 := na
        aEnL := na
        aSlL := na
        aT1L := na
        aT2L := na
        aRewL := na
        aRskL := na

// ———————————————————————————————— OPEN TRADE ZONE (idiom
#2) ————————————————————————
bool openLong  = buySig  and not tActive
bool openShort = sellSig and not tActive
if (openLong or openShort) and showSLTP and not na(atr) and atr > 0
```

```
    bool isLong = openLong
    float entry = close
    float edge  = isLong ? sigBot : sigTop
    if not na(edge)
        float sl = isLong ? edge - atr * slBuf : edge + atr * slBuf
        float risk = math.abs(entry - sl)
        if risk > 0
            float t1 = isLong ? entry + risk * tp1R : entry - risk * tp1R
            float t2 = isLong ? entry + risk * tp2R : entry - risk * tp2R
            color c = isLong ? bullCol : bearCol
            int xr = bar_index + riskExtend
            float rewTop = isLong ? t2 : entry
            float rewBot = isLong ? entry : t2
            float rskTop = isLong ? entry : sl
            float rskBot = isLong ? sl : entry
            aRew := box.new(bar_index, rewTop, xr, rewBot, xloc=xloc.bar_index,
bgcolor=color.new(c, 84), border_color=color.new(c, 45), border_width=1)
            aRsk := box.new(bar_index, rskTop, xr, rskBot, xloc=xloc.bar_index,
bgcolor=color.new(redRisk, 84), border_color=color.new(redRisk, 55), border_width=1)
            aEn  := line.new(bar_index, entry, xr, entry, xloc=xloc.bar_index, color=color.new(c, 0),
width=2)
            aSl  := line.new(bar_index, sl, xr, sl, xloc=xloc.bar_index, color=color.new(redRisk, 10),
width=1, style=line.style_dashed)
            aT1  := line.new(bar_index, t1, xr, t1, xloc=xloc.bar_index, color=color.new(c, 25), width=1)
            aT2  := line.new(bar_index, t2, xr, t2, xloc=xloc.bar_index, color=color.new(c, 45), width=1)
            aEnL := label.new(xr, entry, (isLong ? "LONG @ " : "SHORT @ ") + str.tostring(entry,
format.mintick), xloc=xloc.bar_index, style=label.style_label_left, color=color.new(c, 0),
textcolor=color.white, size=f_size(lblSize))
            aSlL := label.new(xr, sl, "SL @ " + str.tostring(sl, format.mintick), xloc=xloc.bar_index,
style=label.style_label_left, color=color.new(redRisk, 10), textcolor=color.white,
size=f_size(lblSize))
            aT1L := label.new(xr, t1, "TP1 " + str.tostring(tp1R, "#.#") + "R", xloc=xloc.bar_index,
style=label.style_label_left, color=color.new(c, 18), textcolor=color.white, size=f_size(lblSize))
            aT2L := label.new(xr, t2, "TP2 " + str.tostring(tp2R, "#.#") + "R", xloc=xloc.bar_index,
style=label.style_label_left, color=color.new(c, 38), textcolor=color.white, size=f_size(lblSize))
            aRewL := label.new(bar_index, (entry + t2) / 2, "TARGET ZONE", xloc=xloc.bar_index,
style=label.style_label_right, color=color.new(c, 25), textcolor=color.white, size=size.tiny,
text_formatting=text.format_bold + text.format_italic)
            aRskL := label.new(bar_index, (entry + sl) / 2, "RISK ZONE", xloc=xloc.bar_index,
style=label.style_label_right, color=color.new(redRisk, 25), textcolor=color.white, size=size.tiny,
text_formatting=text.format_bold + text.format_italic)
            tActive := true
            tSide := isLong ? 1 : -1
            tBar := bar_index
            tSL := sl
            tT2 := t2

// ———————————————————————————————————————— SIGNAL PILLS (idiom #1)
————————————————————————————————

if showSignals and (buySig or sellSig)
    bool lng = buySig
    float y = lng ? low - atr * 0.6 : high + atr * 0.6
    label pill = label.new(bar_index, y, (lng ? "OB LONG" : "OB SHORT") + "\n" + f_stars(sigGrade)
+ " " + str.tostring(sigGrade, "#.#") + "/10", xloc=xloc.bar_index, yloc=yloc.price, style=lng ?
label.style_label_up : label.style_label_down, color=color.new(lng ? bullCol : bearCol, 0),
textcolor=color.white, size=f_size(lblSize), text_formatting=text.format_bold)
    array.push(sigLbls, pill)
    while array.size(sigLbls) > 40
        label.delete(array.shift(sigLbls))
```

```
// ——————————————————————————————————————— COLORED CANDLES (idiom #3) ———————————————————————————
color barCol = na
if colorCandles
    barCol := buySig ? color.new(bullCol, 20) : sellSig ? color.new(bearCol, 20) : inBullZone ? color.new(bullCol, 72) : inBearZone ? color.new(bearCol, 72) : na
barcolor(barCol, title="OB Zone Candles")

// ——————————————————————————————————————— VWAP + σ BAND (idiom #4) ———————————————————————————
float vwapV = ta.vwap(hlc3)
float vwSd  = ta.stdev(close, vwapLen)
float vwUp  = vwapV + vwapDev * vwSd
float vwLo  = vwapV - vwapDev * vwSd
plot(useVwap ? vwapV : na, "VWAP", color=color.new(chrome, 0), linewidth=2)
pU = plot(useVwap ? vwUp : na, "VWAP +σ", color=color.new(chrome, 65))
pL = plot(useVwap ? vwLo : na, "VWAP -σ", color=color.new(chrome, 65))
fill(pU, pL, color=useVwap ? color.new(chrome, 92) : na, title="VWAP σ Band")

// ——————————————————————————————————————— ALERTS ———————————————————————————————————————————————
alertcondition(buySig,  "OB Bullish Signal", "Order Block Engine [JOAT]: bullish OB retest + reaction on {{ticker}} {{interval}}.")
alertcondition(sellSig, "OB Bearish Signal", "Order Block Engine [JOAT]: bearish OB retest + reaction on {{ticker}} {{interval}}.")

// ——————————————————————————————————————— DASHBOARD (idiom #5) ———————————————————————————————
var table dash = table.new(f_pos(dashPos), 2, 11, frame_color=color.new(accent, 45), frame_width=1, border_width=1, border_color=color.new(#000000, 70))

if barstate.islast and showDash
    int bullN = 0
    int bearN = 0
    float bestGr = 0.0
    float nearDist = na
    float nearLvl = na
    int nearDir = 0
    if array.size(obs) > 0
        for i = 0 to array.size(obs) - 1
            OB ob = array.get(obs, i)
            if not ob.mit
                if ob.dir == 1
                    bullN += 1
                else
                    bearN += 1
                bestGr := math.max(bestGr, ob.grade)
                float d = close > ob.top ? close - ob.top : close < ob.bot ? ob.bot - close : 0.0
                if na(nearDist) or d < nearDist
                    nearDist := d
                    nearLvl := ob.dir == 1 ? ob.top : ob.bot
                    nearDir := ob.dir

    ps = f_size(dashSize)
    string biasTxt = biasDir == 1 ? "BULLISH" : biasDir == -1 ? "BEARISH" : "NEUTRAL"
    color  biasC   = biasDir == 1 ? bullCol : biasDir == -1 ? bearCol : txtLo
    string nearTxt = na(nearLvl) ? "—" : str.tostring(nearLvl, format.mintick)
    color  nearC   = nearDir == 1 ? bullCol : nearDir == -1 ? bearCol : txtHi
    string distTxt = na(nearDist) ? "—" : nearDist == 0 ? "INSIDE" : str.tostring(100.0 * nearDist / close, "#.##") + "%"
```

```
    string bestTxt = bestGr > 0 ? f_stars(bestGr) + " " + str.tostring(bestGr, "#.#") + " · " +
f_tier(bestGr) : "—"
    string lastMitTxt = lastMitDir == 0 ? "—" : (lastMitDir == 1 ? "BULL " : "BEAR ") +
(na(lastMitLvl) ? "" : str.tostring(lastMitLvl, format.mintick)) + (na(lastMitGr) ? "" : " ★" +
str.tostring(lastMitGr, "#.#"))
    color  lastMitC = lastMitDir == 1 ? bullCol : lastMitDir == -1 ? bearCol : txtLo
    string sigTxt = buySig ? "BUY ▲" : sellSig ? "SELL ▼" : tActive ? (tSide == 1 ? "LONG OPEN" :
"SHORT OPEN") : "—"
    color  sigC   = buySig ? bullCol : sellSig ? bearCol : tActive ? accent : txtLo

    table.cell(dash, 0, 0, "ORDER BLOCK ENGINE", text_color=hdrDark, text_size=ps,
text_halign=text.align_left, bgcolor=accent)
    table.cell_set_text_formatting(dash, 0, 0, text.format_bold + text.format_italic)
    table.cell(dash, 1, 0, "OB [JOAT]", text_color=hdrDark, text_size=ps,
text_halign=text.align_center, bgcolor=chrome)
    table.cell_set_text_formatting(dash, 1, 0, text.format_bold + text.format_italic)

    table.cell(dash, 0, 1, "Bias", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelBg)
    table.cell(dash, 1, 1, biasTxt, text_color=biasC, text_size=ps, text_halign=text.align_center,
bgcolor=panelBg)

    table.cell(dash, 0, 2, "Bullish OBs", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelAlt)
    table.cell(dash, 1, 2, str.tostring(bullN), text_color=bullCol, text_size=ps,
text_halign=text.align_center, bgcolor=panelAlt)

    table.cell(dash, 0, 3, "Bearish OBs", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelBg)
    table.cell(dash, 1, 3, str.tostring(bearN), text_color=bearCol, text_size=ps,
text_halign=text.align_center, bgcolor=panelBg)

    table.cell(dash, 0, 4, "Nearest OB", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelAlt)
    table.cell(dash, 1, 4, nearTxt, text_color=nearC, text_size=ps, text_halign=text.align_center,
bgcolor=panelAlt)

    table.cell(dash, 0, 5, "Distance", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelBg)
    table.cell(dash, 1, 5, distTxt, text_color=txtHi, text_size=ps, text_halign=text.align_center,
bgcolor=panelBg)

    table.cell(dash, 0, 6, "Strongest OB", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelAlt)
    table.cell(dash, 1, 6, bestTxt, text_color=accent, text_size=ps, text_halign=text.align_center,
bgcolor=panelAlt)

    table.cell(dash, 0, 7, "Last Mitigated", text_color=txtLo, text_size=ps,
text_halign=text.align_left, bgcolor=panelBg)
    table.cell(dash, 1, 7, lastMitTxt, text_color=lastMitC, text_size=ps, text_halign=text.align_center,
bgcolor=panelBg)

    table.cell(dash, 0, 8, "Active Signal", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelAlt)
    table.cell(dash, 1, 8, sigTxt, text_color=sigC, text_size=ps, text_halign=text.align_center,
bgcolor=panelAlt)

    table.cell(dash, 0, 9, "Signals B / S", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelBg)
```

```
    table.cell(dash, 1, 9, str.tostring(totBuy) + " / " + str.tostring(totSell), text_color=txtHi,
text_size=ps, text_halign=text.align_center, bgcolor=panelBg)

    table.cell(dash, 0, 10, "Trades W / L", text_color=txtLo, text_size=ps, text_halign=text.align_left,
bgcolor=panelAlt)
    table.cell(dash, 1, 10, str.tostring(wins) + " / " + str.tostring(losses), text_color=txtHi,
text_size=ps, text_halign=text.align_center, bgcolor=panelAlt)
```